AARHUS
UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Computer Architecture

Homework Problems for section
Memory Hierarchy

Upload one solution per group to Brightspace!

TOPICS COVERED

These homework tasks focus on the important concepts about the memory hierarchy, which has been introduced in the past three lectures about the following topics:

- Storage technologies
- Spatial and temporal locality
- Caching
- Virtual memory
- Malloc

QUESTIONS

Answer the following questions briefly (1-3 sentences)

1. What is the difference between volatile and nonvolatile memory?
2. Why does it take a long time to read or write a value in the main memory?
3. What's the easiest way for a HDD manufacturer to increase the capacity of a drive?
4. Why does the CPU not wait for a disk read? How does it get the data instead?
5. Name a few advantages and disadvantages of SSD over HDD. What can we do about the disadvantages?
6. Define temporal and spatial locality. How do they help with slow memory and disk access times?
7. What is the difference between the 3 cache miss types?
8. Approximately how many clock cycles does it take to access data from the

following memories: registers, CPU caches, main memory, disk, network storage.

9. How do we read data from a direct mapped cache (step by step)? At which step do we know for sure if we have a cache hit or miss?
10. What is the “memory mountain”?
11. Why is virtual memory considered a cache?
12. What is a Page Table and a Page Fault? How are Page Faults handled?
13. What problem is solved by the Translation Lookaside Buffer (TLB)?
14. In what cases do we get a Segmentation Fault?
15. How does *malloc* work in general, and what constraints must be satisfied by its implementations (allocators)?
16. What rules must be followed by a C programmer when using *malloc*? What happens when the rules are violated?
17. What is internal and external fragmentation in memory allocation?

PRACTICAL PROBLEMS

1. Data Cache

Design a 16 byte cache over 32 bytes of main memory, that produces at least 2 hits with the following address trace: 0, 3, 14, 25, 28, 14. Prove that your solution satisfies the goal by showing the cache contents after each read, and marking cache hit or miss!

2. Cache miss rate

Consider a memory system that consists of two cache layers L1 and L2 above the main memory.

- What is the average access time of a memory address, in CPU cycles?
  Use the values from the table below to determine the cache hit rates.
- Based on your calculations give the general formula of access time for this 2-level cache configuration.

A = 6th digit of your AU ID (AUID=123456 → A=6)
B = 5th digit of your AU ID (AUID=123456 → B=5)

A	L1 hit rate	B	L2 hit rate
1, 2 or 3	99%	1, 2 or 3	70%

4 or 5	98%	4 or 5	65%
6 or 7	96%	6 or 7	60%
8 or 9	95%	8 or 9	55%

L1 access time: 1 cycle
L2 access time: 10 cycles
Memory access time: 100 cycles

PROGRAMMING EXERCISE

Consider the simple program below that tests the memory allocator in your computer. It tries allocating more and more integers in each iteration until we run out of memory and the program crashes.

```
#include <stdlib.h> // For malloc
#include <stdio.h> // For printf

int main(){

    printf("Integers are %d bytes in this computer\n", (int) sizeof(int));

    unsigned long long i=0;
    while(1){

        // Try to allocate 'i' million ints
        void *memory = malloc(i * 1E6 * sizeof(int));

        if(memory == NULL){
            // malloc failed to allocate the memory we asked for
            return -1;
        }

        // 'llu' is for 'long long unsigned'
        printf("Successfully allocated %llu million integers\n", i);
        ++i;
    }

    return 0;
}
```

Start by understanding the code and running it on your machine. Save it as **main.c** and compile with the following command: **gcc main.c -o memory_test** This produces an executable file called *memory_test* in the same folder. Run it by **./memory_test** to see the result.

On a 2019 MacBook Pro it produces the following output:

```
$ gcc main.c -o malloc_test && ./malloc_test
Integers are 4 bytes in this computer
Successfully allocated 0 million integers
Successfully allocated 1 million integers
...
Successfully allocated 8387 million integers
malloc_test(69516,0x10938edc0) malloc: can't allocate region
*** mach_vm_map(size=33552003072) failed (error code=3)
malloc_test(69516,0x10938edc0) malloc: *** set a breakpoint in malloc_error_break to
debug
```

Exercises:

a. Use ***calloc*** instead of *malloc* (calloc overwrites the newly allocated block with zeros), and check the time penalty (e.g. what's the total runtime of your program with malloc and calloc).
b. Use ***realloc*** instead of *malloc* to increase the allocated memory instead of allocating a new block, and see if there's a difference in the maximum amount of allocated memory!
c. Use ***malloc*** but also ***free*** the memory within the same iteration, before trying to allocate a larger block. See if there's a difference in runtime or the maximum amount of allocated memory.